

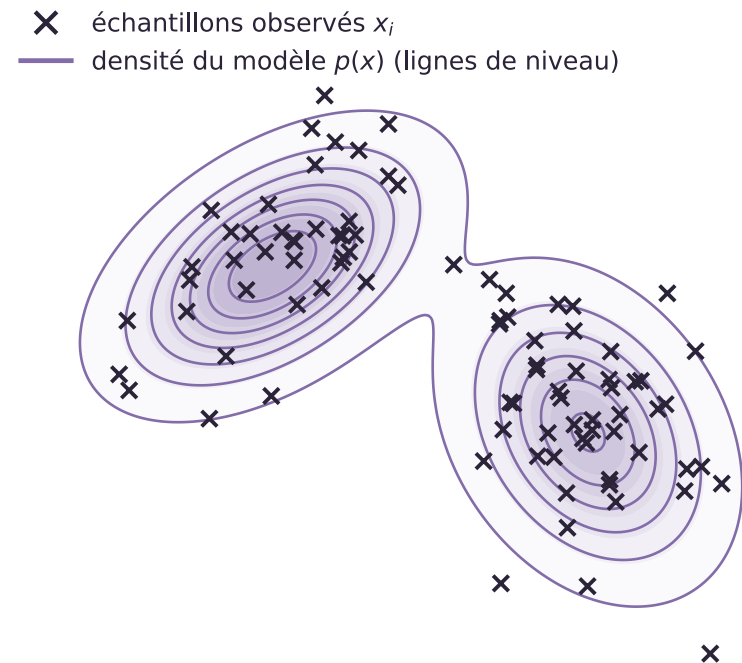
Deep Learning

6. Réseaux de neurones génératifs

Romain Tavenard

Les modèles génératifs en un coup d'œil

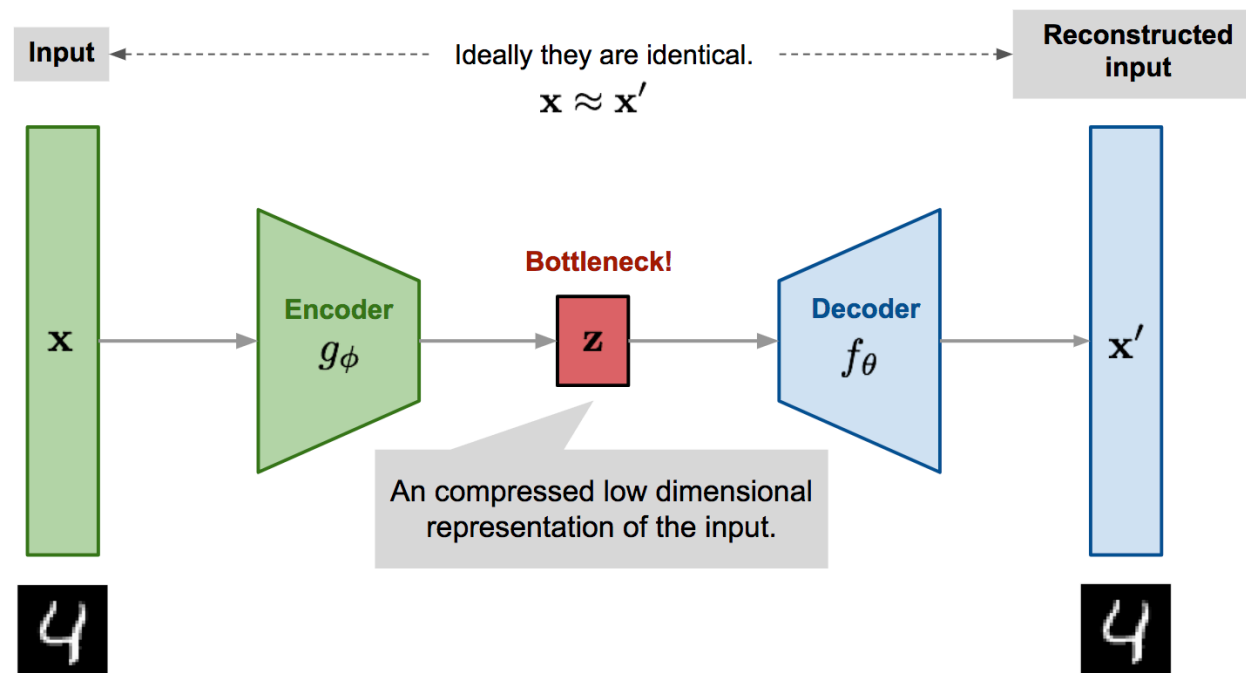
- Objectif : modéliser $p(x)$ ou $p(x|y)$
 - explicitement
 - ex. modèles de mélange gaussien
 - **implicitement**
 - au moins permettre l'échantillonnage (c'est-à-dire générer de nouvelles données)
 - ex. auto-encodeurs variationnels, réseaux antagonistes génératifs



GMM à 2 composantes

Auto-encodeurs [Hinton & Salakhutdinov, 2006]

- Encodent l'information dans un espace latent
 - typiquement de dimension plus faible
 - dans le même esprit qu'une ACP non linéaire
 - pas un modèle génératif *per se* !



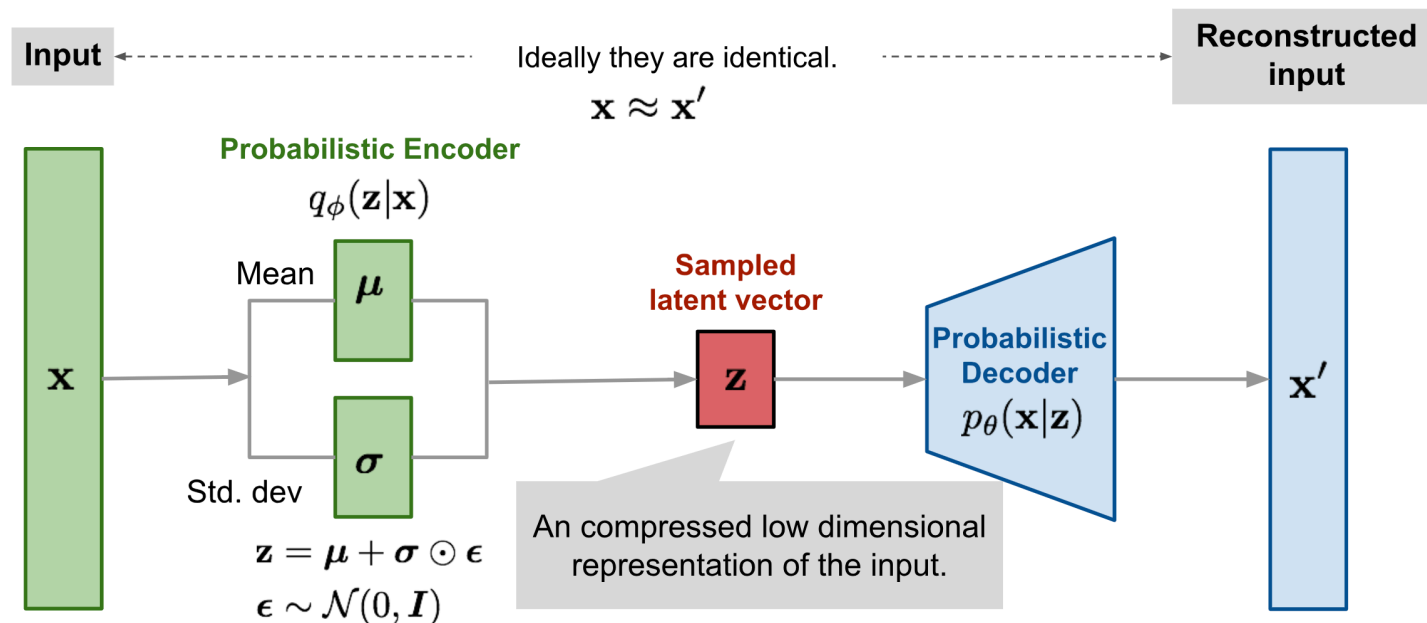
Source : lilianweng.github.io

Auto-encodeurs variationnels [Kingma & Welling, 2014]

- Transformer les auto-encodeurs en modèles génératifs
- Une loi $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ sur l'espace latent, imposée par :

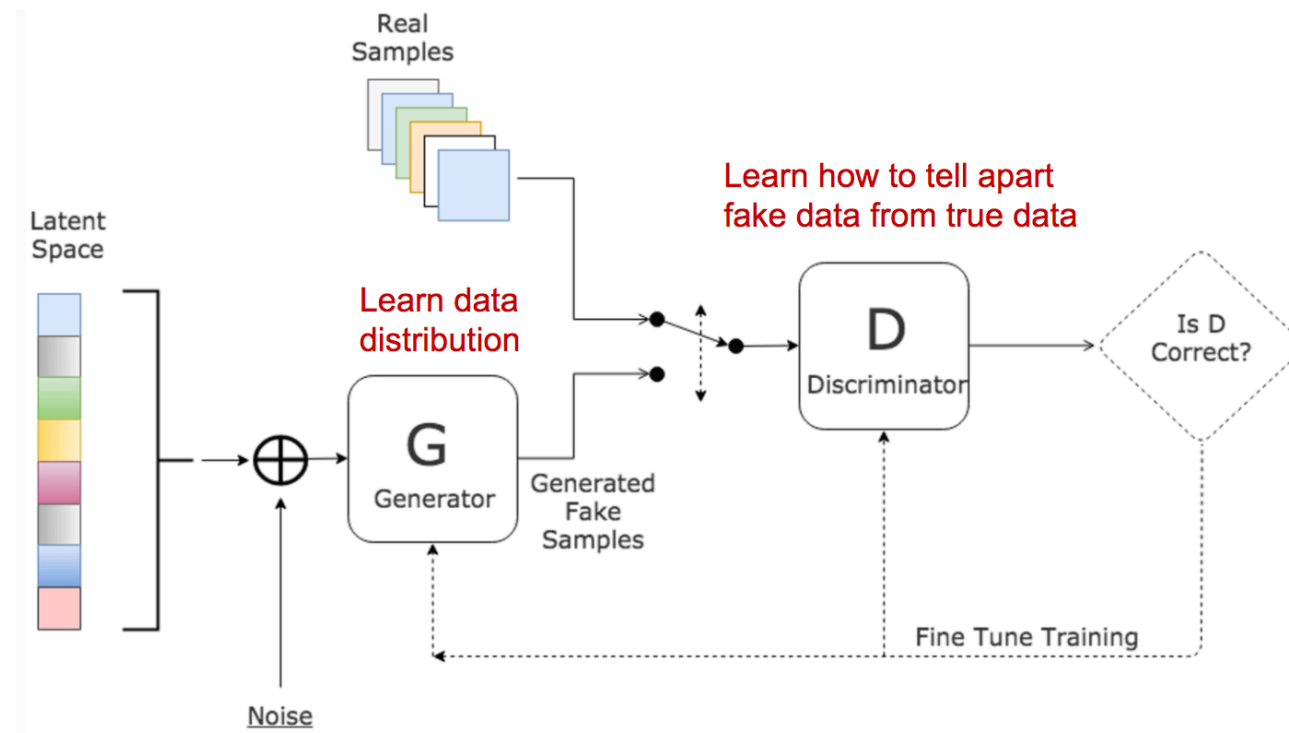
$$\mathcal{L} = \underbrace{-\log p_{\theta}(\mathbf{x}|\mathbf{z})}_{\text{reconstruction}} + \underbrace{\text{KL}(q_{\Phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{rester proche de l'a priori}}$$

- Processus génératif** : tirer $\mathbf{z}$ depuis l'a priori $\rightarrow$ décoder



Source : lilianweng.github.io

Les GAN [Goodfellow et al., 2014]



Source : *KDNuggets.com*

- Deux réseaux s'affrontent
 - D (discriminateur) : produit $P(\text{l'échantillon est réel})$
 - G (générateur) : transforme du bruit en faux échantillons
 - D veut avoir raison, G veut tromper $D \rightarrow$ **min-max**

Les GAN [Goodfellow et al., 2014]

- Fonction de perte :

$$\min_G \max_D L(D, G) = \underbrace{\mathbb{E}_{x \sim p_r(x)} [\log D(x)]}_{\text{D a raison (réel)}} + \underbrace{\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]}_{\text{D a raison (faux) / G veut } \downarrow}$$

- ▶ $\max_D$: pousse les deux termes vers le haut (discriminateur plus tranchant)
- ▶ $\min_G$: pousse le 2e terme vers le bas ($D(G(z)) \rightarrow 1$, les faux semblent réels)
- Optimisation : Alternier l'entraînement de G et de D
 - ▶ Très instable en pratique
- Processus génératif
 - ▶ Tirer z selon p_z (bruit aléatoire)
 - ▶ Le passer au générateur pour calculer $G(z)$ (échantillon factice)

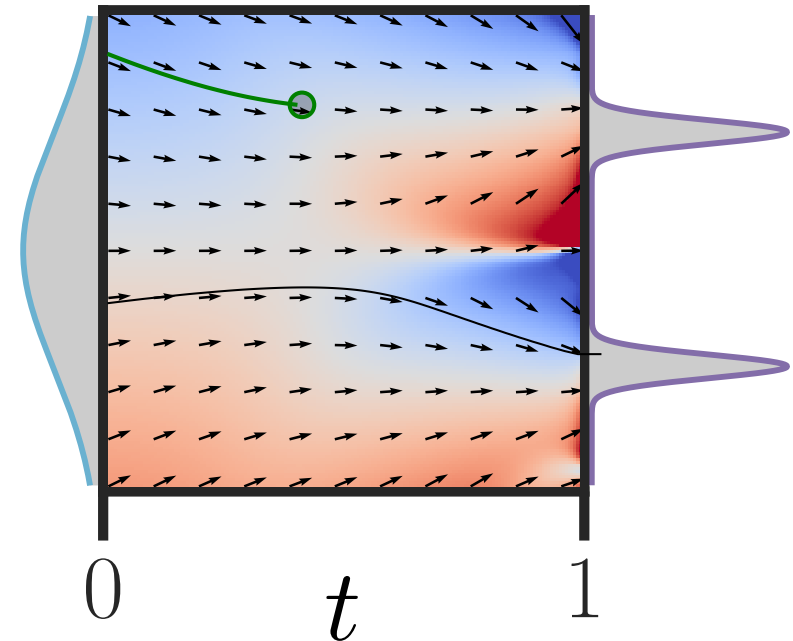
Conditional Flow Matching [Lipman et al., 2023]

- **Principe** : apprendre un champ de vecteurs transportant le bruit ($t=0$) vers les données ($t=1$) en minimisant

$$\mathbb{E}_{x_0, x_1, t} \| u^\theta(x_t, t) - (x_1 - x_0) \|^2$$

- Au moment de la génération, utiliser un solveur d'équation différentielle ordinaire (par ex. schéma d'Euler) :

$$x_{t+\varepsilon} \leftarrow x_t + \varepsilon u^\theta(x_t, t)$$



Le champ de vitesse appris transporte le bruit ($t=0$) vers les données ($t=1$).

Source : Emonet et al. "A Visual Dive into Conditional Flow Matching", ICLR

2025